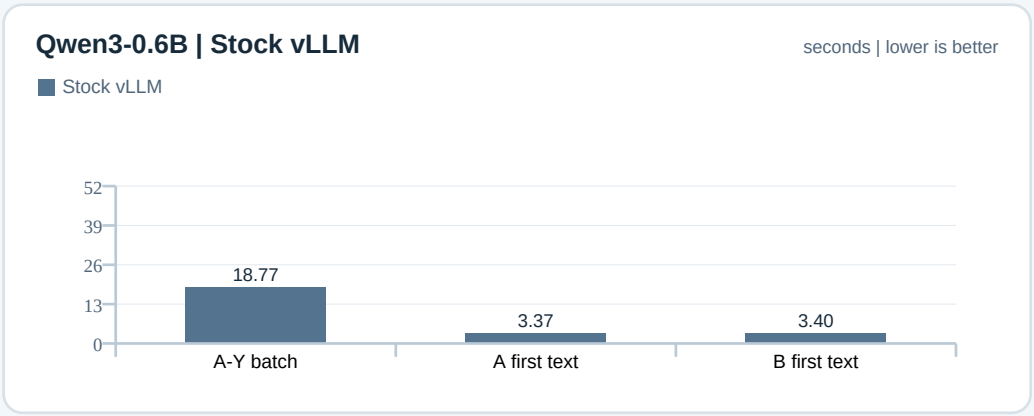
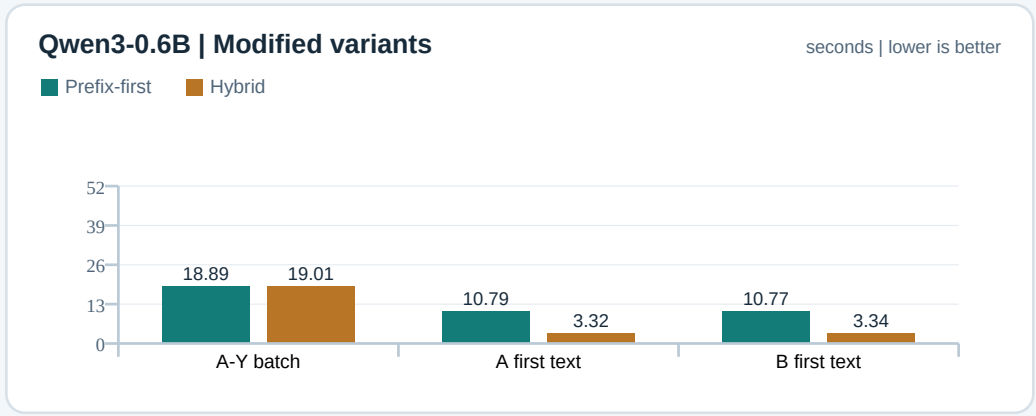
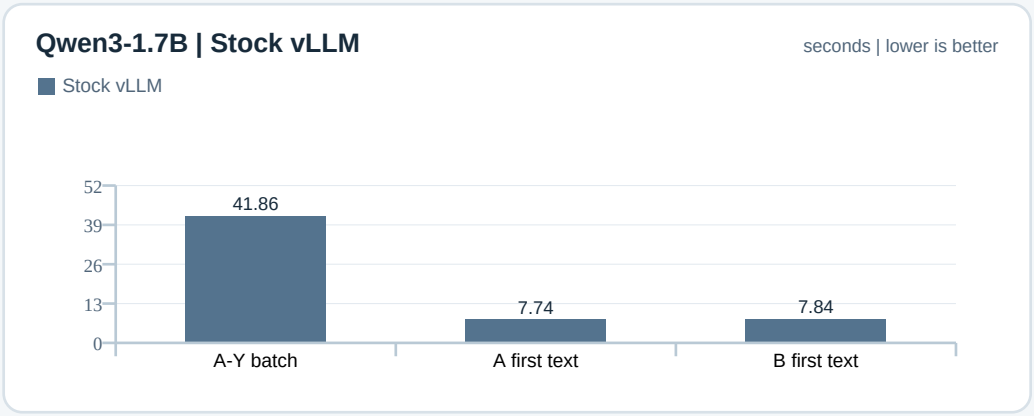
Prefix-aware request scheduling

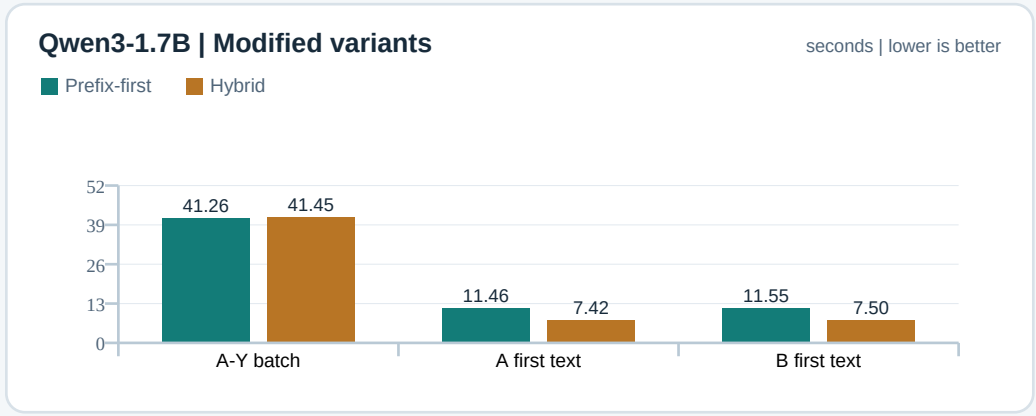
What changed. Rank waiting requests by reusable cached prefix; break ties by FCFS. A hybrid protects the first three requests in FCFS order and applies prefix ranking to the rest, with a waiting-time safeguard.

Original concurrent A-Y workload | 25 candidates x 64 outputs | 3 running slots | 3 timing rounds per policy | matching 1.7B replication completed









Observed result. Batch seconds (FCFS / prefix-first / hybrid): 0.6B 18.77 / 18.89 / 19.01; 1.7B 41.86 / 41.26 / 41.45. On 1.7B, A first text was 7.74s / 11.46s / 7.42s.

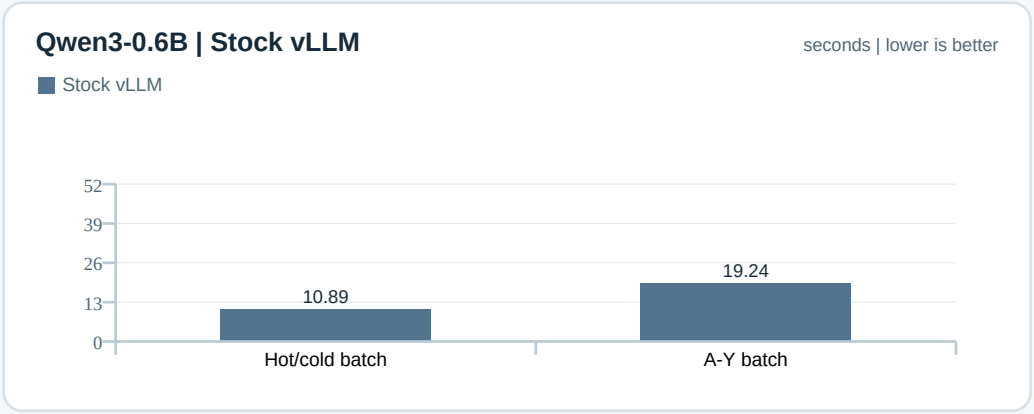
Interpretation. Prefix-first delayed A/B on both models; the hybrid protected their early position. Close group medians and three timing rounds do not establish a general batch-speed advantage. The same ten-second wait safeguard applies on both models.

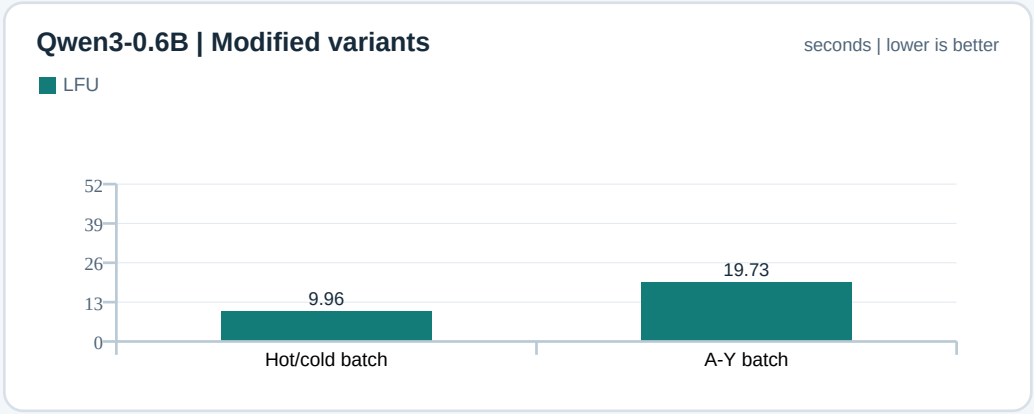
Sources: experiments/01-prefix-scheduling/RESULTS.txt | experiments/01-prefix-scheduling/qwen3-1.7B/RESULTS.txt

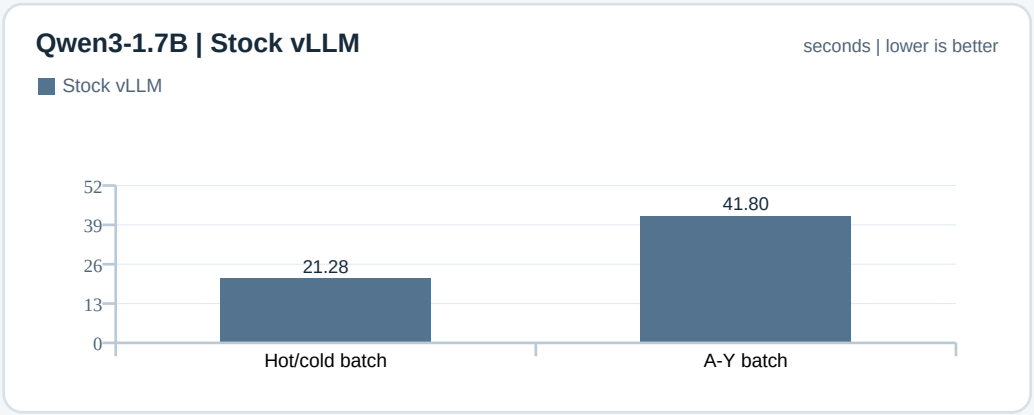
Frequency-based cache retention

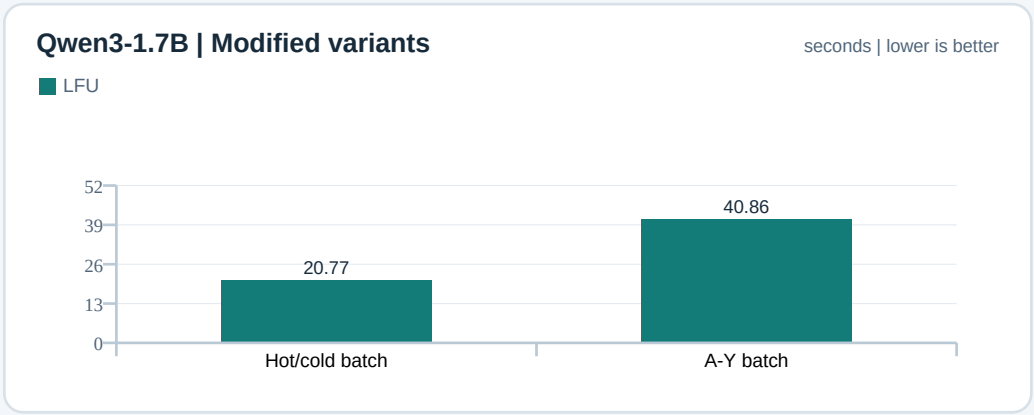
What changed. Prefer evicting less frequently reused, zero-reference cached blocks. Keep actively referenced blocks ineligible for eviction and preserve FCFS request order.

Two profiles: 42 sequential hot/cold requests and original concurrent A-Y | 3 timing rounds per policy/profile | matching 1.7B replication completed









Observed result. Hot-return reuse increased from 16 to 272 tokens on both models. Probe first text: 0.6B 119.50 to 52.54ms; 1.7B 291.08 to 88.32ms. A-Y batch-median changes: +2.5% on 0.6B, -2.25% on 1.7B.

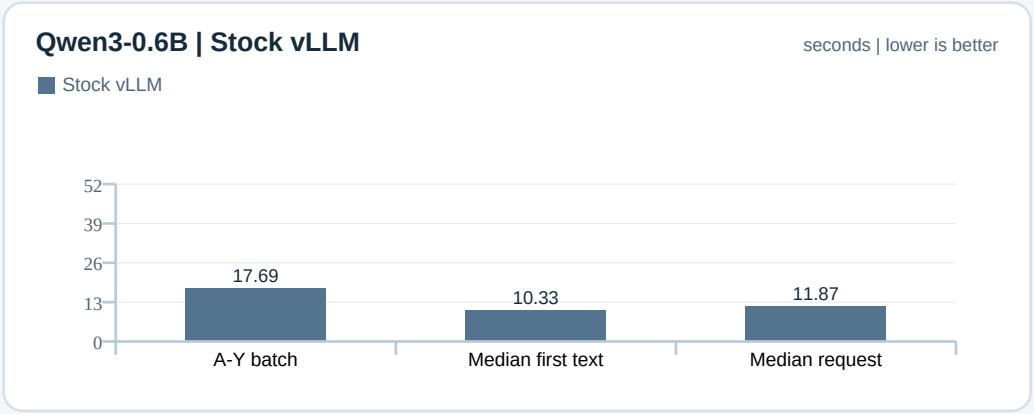
Interpretation. A-Y reused the same 2,432 candidate tokens under both policies on both models; its timing differences are not evidence of extra reuse. The no-reuse control also varied, particularly on 0.6B. Strong retention/probe results do not establish a universal full-batch gain.

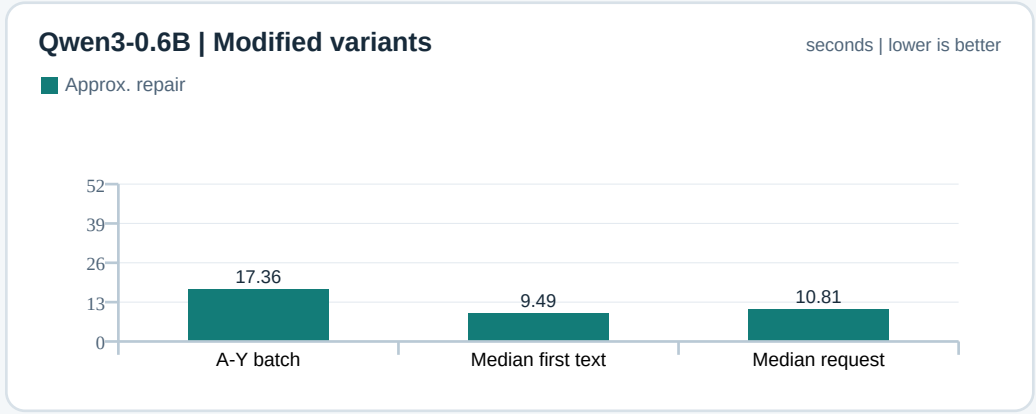
Sources: experiments/02-frequency-eviction/RESULTS.txt | experiments/02-frequency-eviction/RESULTS-AY.txt | experiments/02-frequency-eviction/qwen3-1.7B/RESULTS.txt

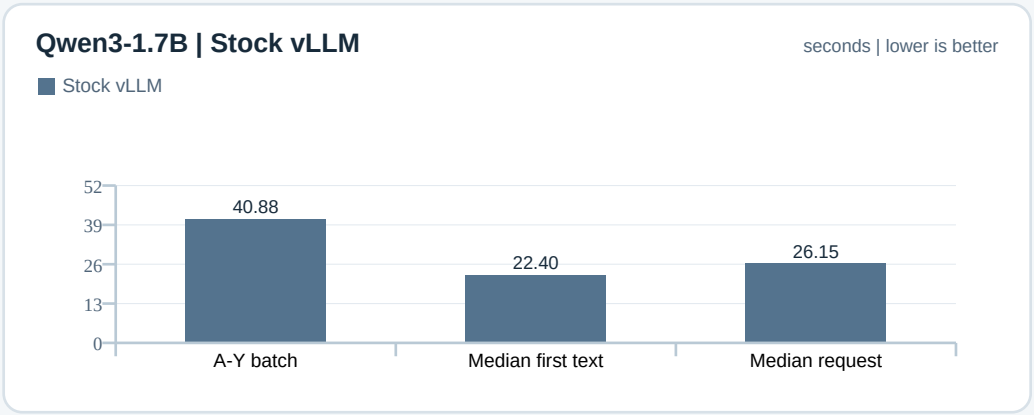
Middle-block reuse with partial repair

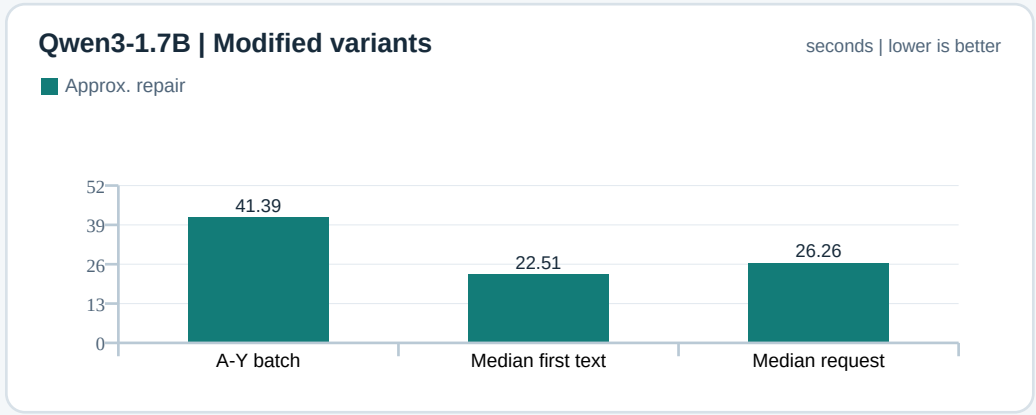
What changed. Match token pages even when earlier context differs, reuse their KV approximately, and recompute a boundary subset. The Metal worker skips selected rows using an alternate attention path; this is not exact causal-prefix caching.

Original A-Y | 25 candidates + 3 blockers | 96 KV pages | 3 slots | 3 matched timing pairs | matching 1.7B replication completed









Observed result. Actual reuse skipped 308 rows/layer/run on both models. 0.6B repair lost 2/3 time pairs despite a 1.85% lower group median. On 1.7B, batch median changed +1.25%; candidate exact matches were [14, 14, 14]/25 across the three pairs (0.6B: 10/25 each).

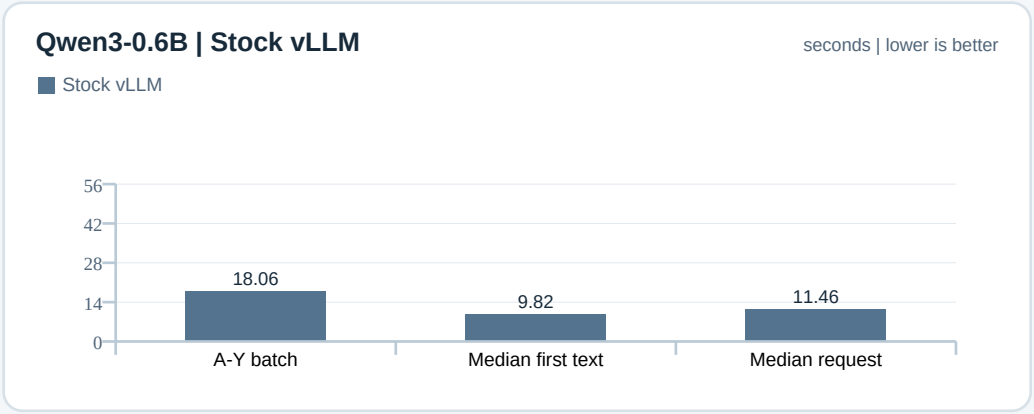
Interpretation. No reliable performance or quality advantage is established. Matching middle-page tokens with different preceding context is approximate. Changed answers can also reflect the alternate attention path; exact agreement is not a semantic quality score.

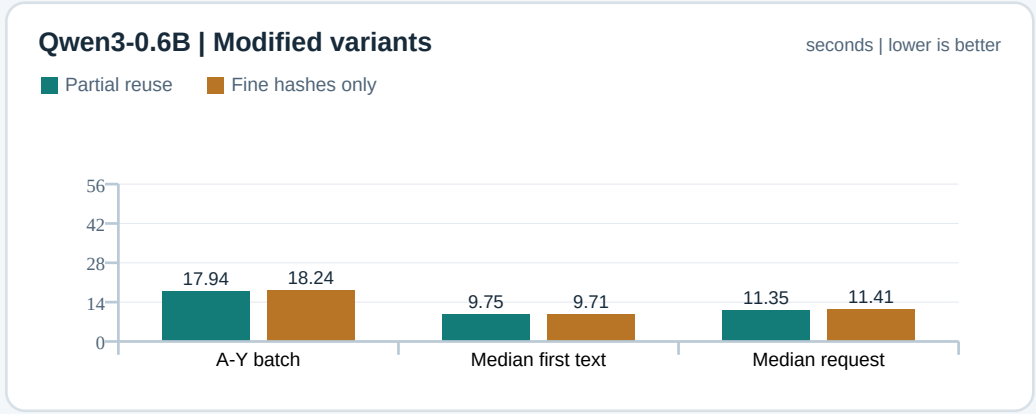
Sources: experiments/03-nonprefix-repair/RESULTS-VLLM-AY.txt | experiments/03-nonprefix-repair/qwen3-1.7B/RESULTS.txt

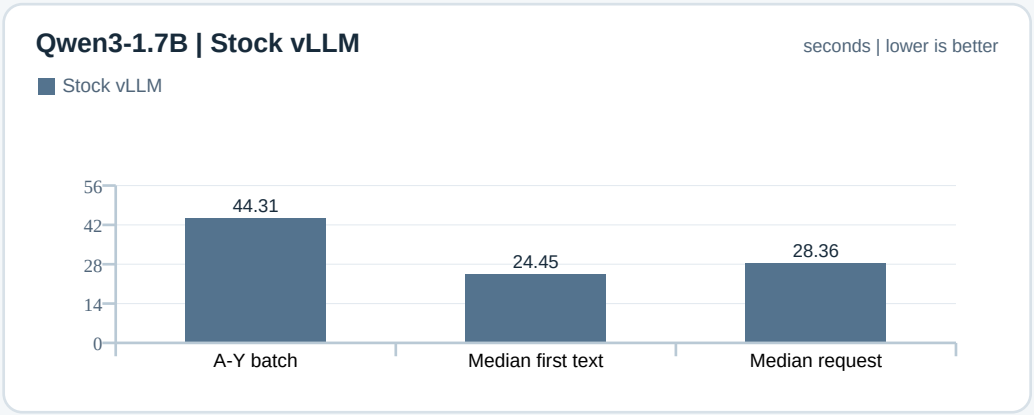
Exact partial-prefix caching

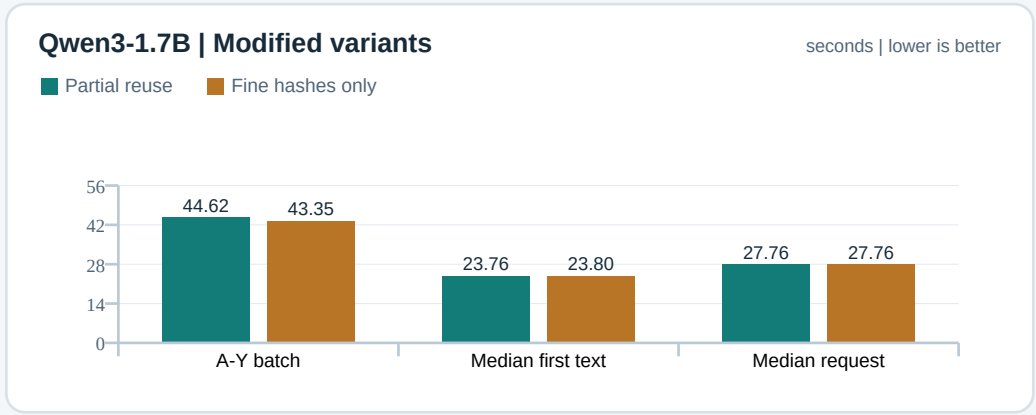
What changed. Index matching prefixes inside a 16-token physical page. Reuse the exact causal prefix and copy the partial page before continuing, preserving page ownership and invalidation. Physical pages remain 16 tokens.

Original A-Y | same native Metal worker | baseline, fine-hash control and partial reuse | 3 rotated rounds | matching 1.7B replication completed









Observed result. Candidate reuse increased 2,432 to 2,563 tokens on both models. Batch-median change: -0.64% on 0.6B; +0.70% on 1.7B. Partial won 3/3 and 2/3 paired rounds respectively. The 31-of-32-token probe matched cold generation on both models.

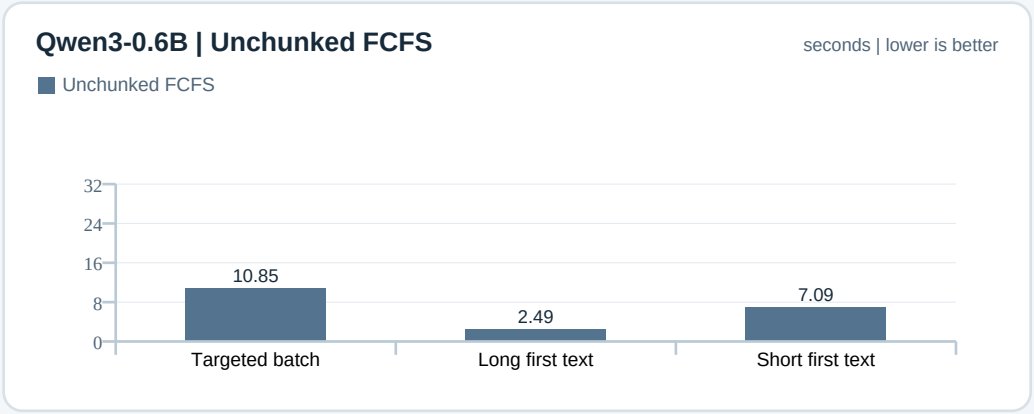
Interpretation. The 1.7B partial answers matched stock in [18, 18, 18]/25 candidates by round; 0.6B matched 16/25 each. Exact causal caching can change low-precision prefill shapes and generation. Small, overlapping timing ranges and three repeats do not establish a general speedup or semantic quality parity.

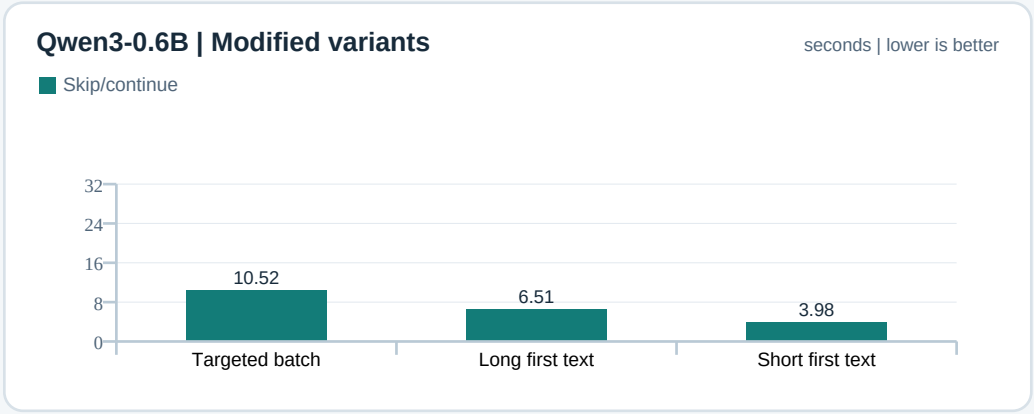
Sources: experiments/04-partial-prefix/RESULTS.txt | experiments/04-partial-prefix/qwen3-1.7B/RESULTS.txt

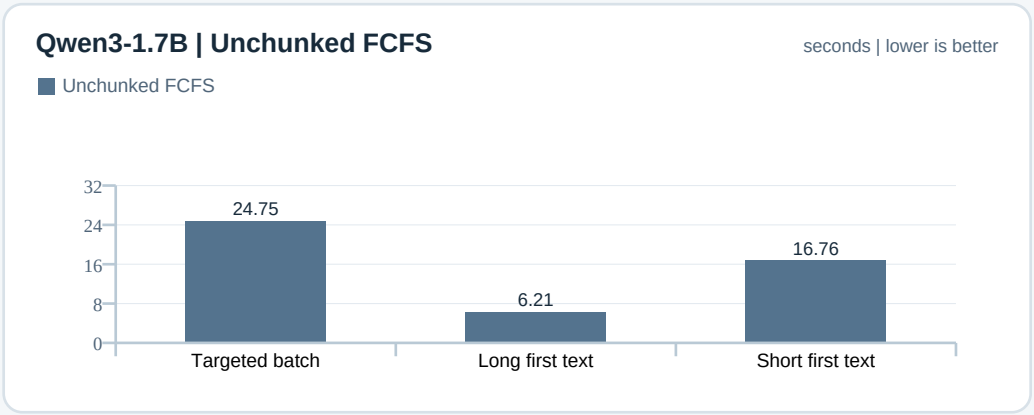
Bounded skip-and-continue scheduling

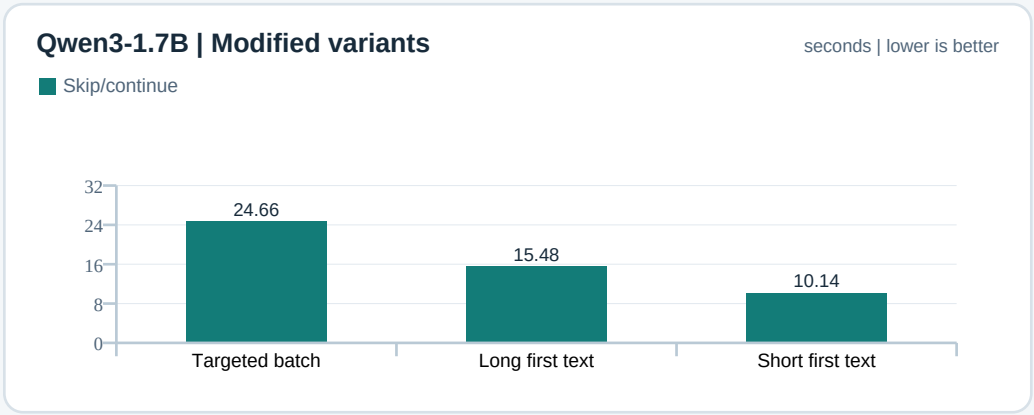
What changed. With chunked prefill disabled, bypass a waiting head that cannot fit the remaining token budget and admit a later fitting request. Restore skipped order; bound the scan to 16 requests, four bypasses/request and 30-second age protection.

Targeted A-Y: 13 long then 12 short prompts | 8 blockers/slots | 512-token batch | 3 matched pairs | matching 1.7B replication completed









Observed result. Batch-median change: -3.04% on 0.6B; -0.33% on 1.7B. On 1.7B, long first text rose 6.21s to 15.48s; short first text fell 16.76s to 10.14s. The long/short tradeoff repeated on both models.

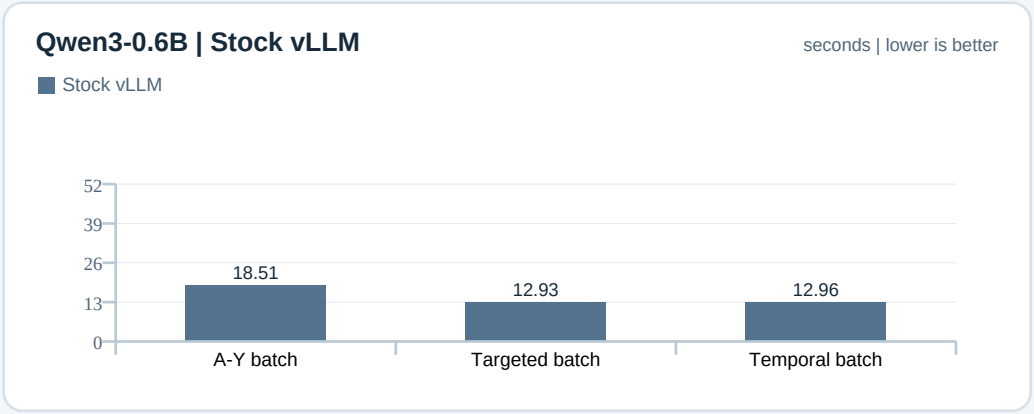
Interpretation. Original A-Y did not activate skipping on either model; targeted packing did. The 1.7B batch gain is tiny and its pairs were mixed. Four-bypass and age bounds limit bypassing, but do not free occupied slots or guarantee an admission deadline. Semantic quality was not scored.

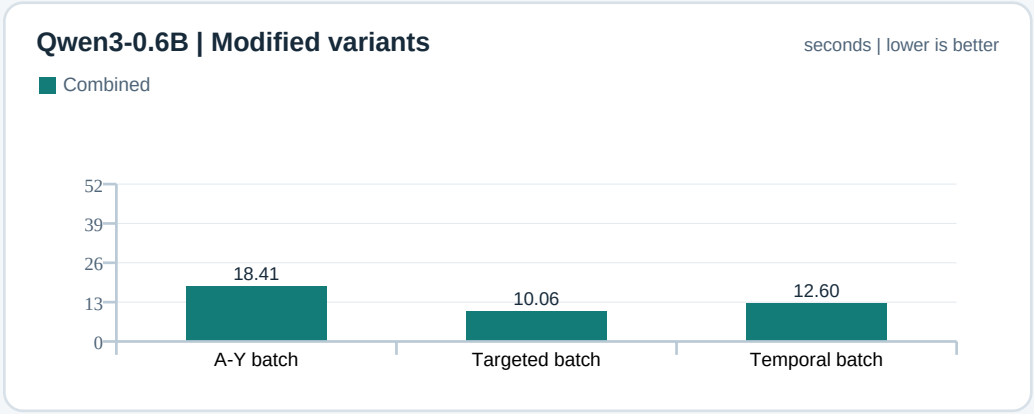
Sources: experiments/05-skip-continue/RESULTS.txt | experiments/05-skip-continue/qwen3-1.7B/RESULTS.txt

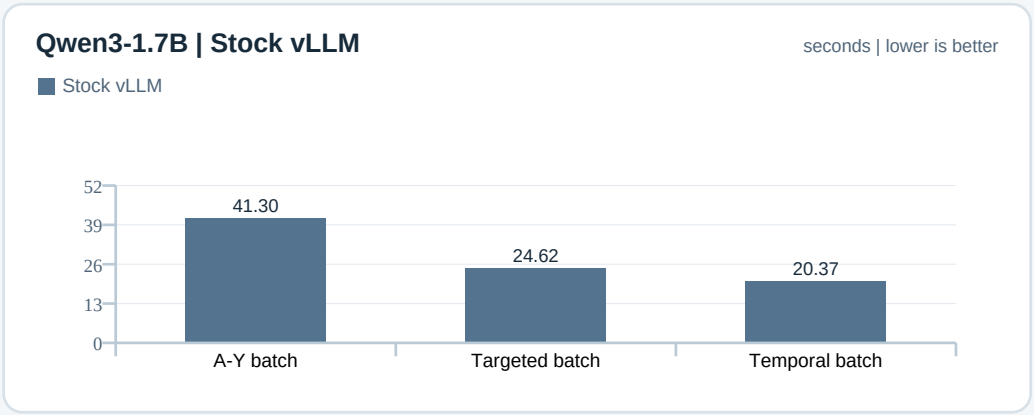
Combined cache and scheduling policies

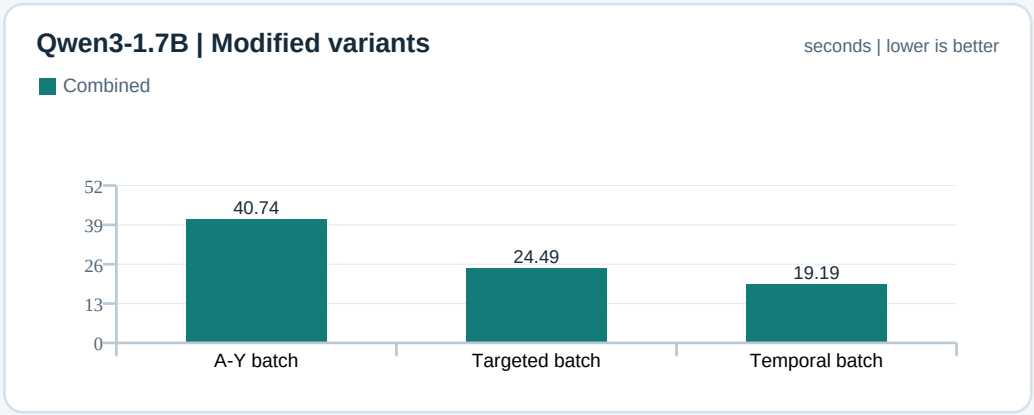
What changed. Enable experiment 02 frequency retention, experiment 04 exact partial-prefix reuse, and experiment 05 bounded skip/continue together. Keep approximate middle-block repair disabled; both controls use unchunked prefill.

A-Y, targeted and temporal profiles | both models | 24 fresh-server runs/model | 3 timing pairs/profile









Observed result. Batch-median changes (A-Y / targeted / temporal): 0.6B -0.55% / -22.21% / -2.79%; 1.7B -1.37% / -0.53% / -5.78%. Combined won 5/9 paired rounds on 0.6B and 7/9 on 1.7B. Extra prefix reuse activated in all three profiles.

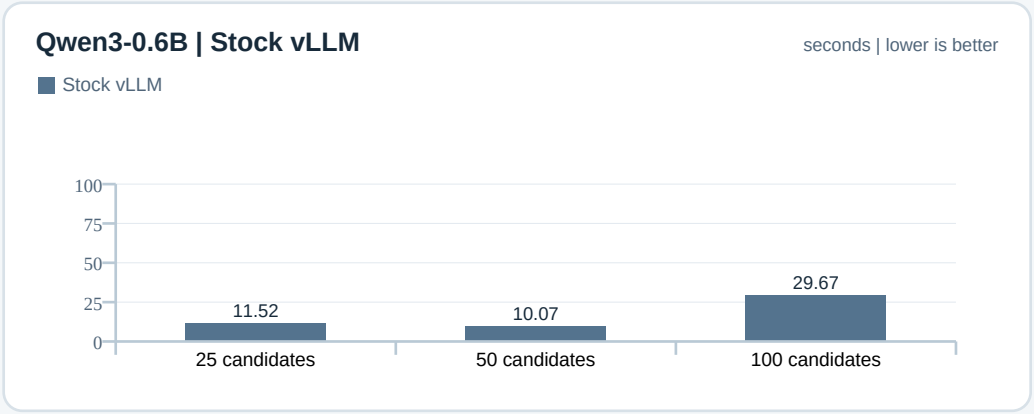
Interpretation. A ratio of group medians is not the median paired effect; retain the paired evidence. Targeted scheduling shifts latency toward long prompts. Combining policies does not isolate each contribution, and these small synthetic comparisons do not establish a generally faster scheduler.

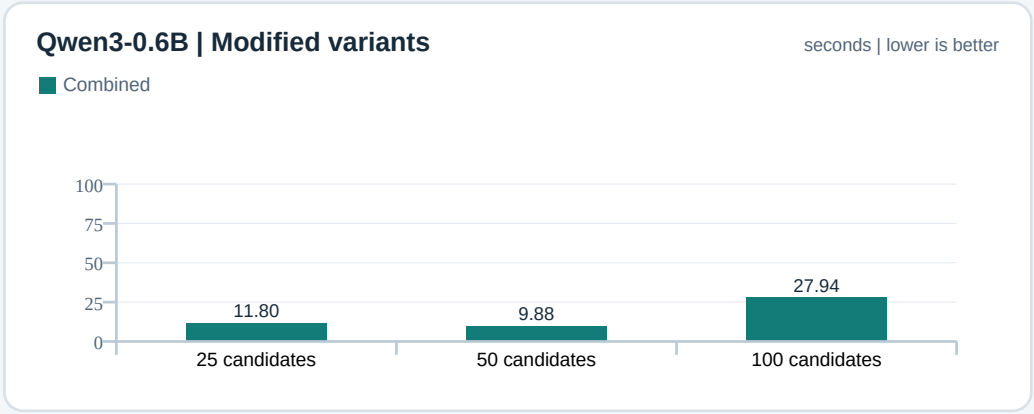
Sources: experiments/06-combined/RESULTS.txt | experiments/06-combined/qwen3-1.7B/RESULTS.txt

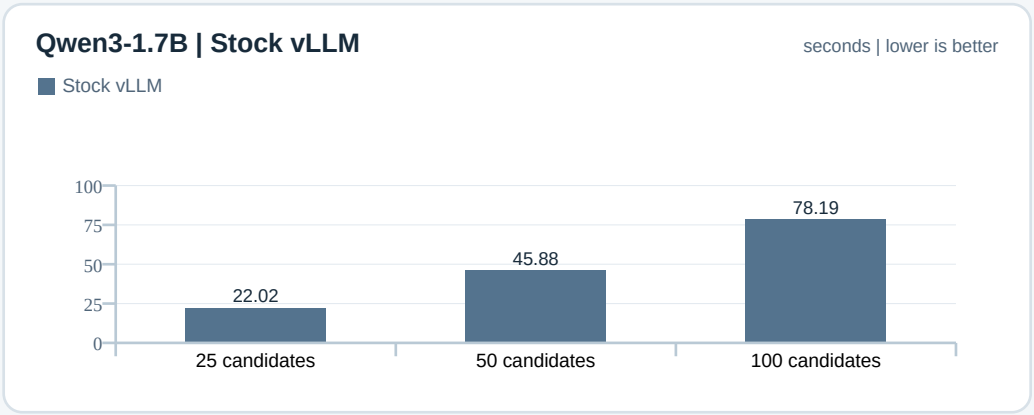
Request-count and model-size scaling

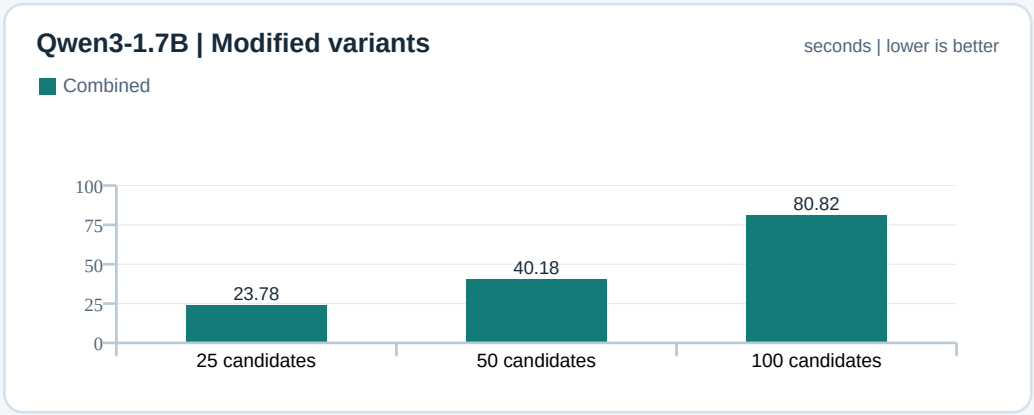
What changed. Scale the combined policy from experiment 06 to 25, 50 and 100 candidates on Qwen3-0.6B and Qwen3-1.7B. This changes workload scale and model size; it introduces no new policy.

Both pinned models | 320/64-token long/short prompts | 8 slots | 256 pages | 48 runs | 3 pairs/scale point









Observed result. Combined was faster in only 8/18 matched pairs. Cached-token gains repeated on both models: +212, +441 and +885 at 25/50/100 candidates. The 1.7B/50 group median improved 12.43%, but 1.7B/100 regressed 3.37%.

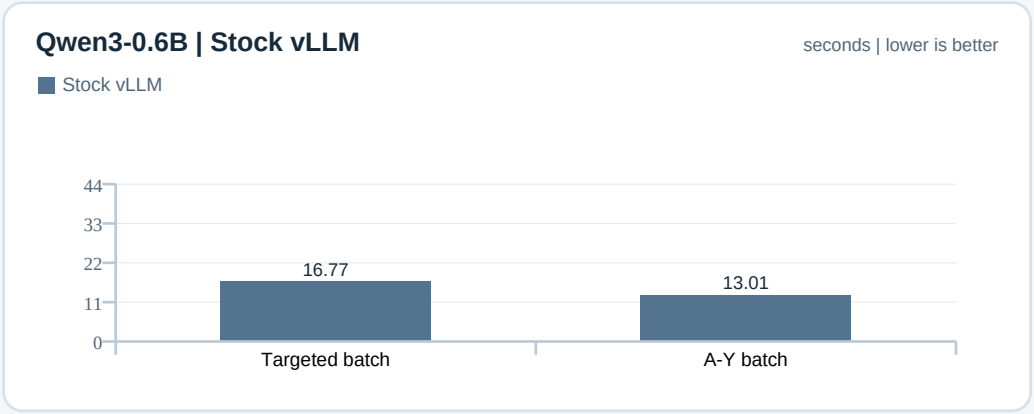
Interpretation. Run variability is large: 0.6B/50 was faster in absolute time than 0.6B/25. These bars are observed medians, not a clean scaling law or proof of a generally faster scheduler.

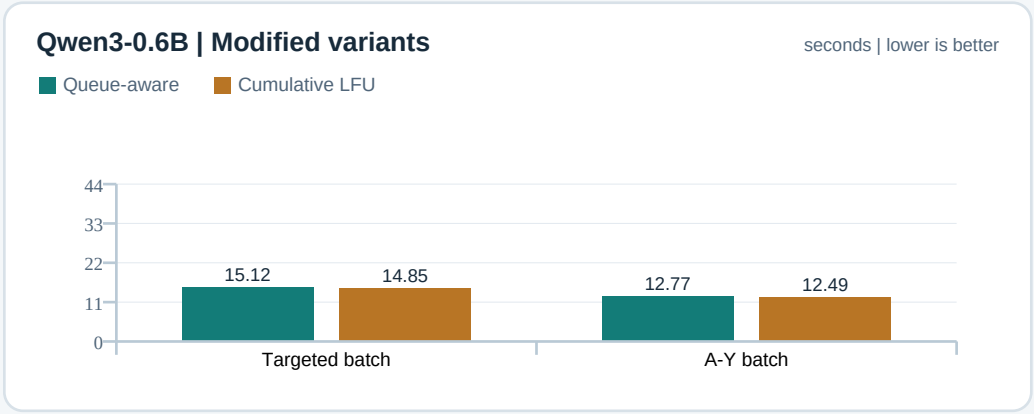
Sources: experiments/07-scale-and-model/RESULTS.txt

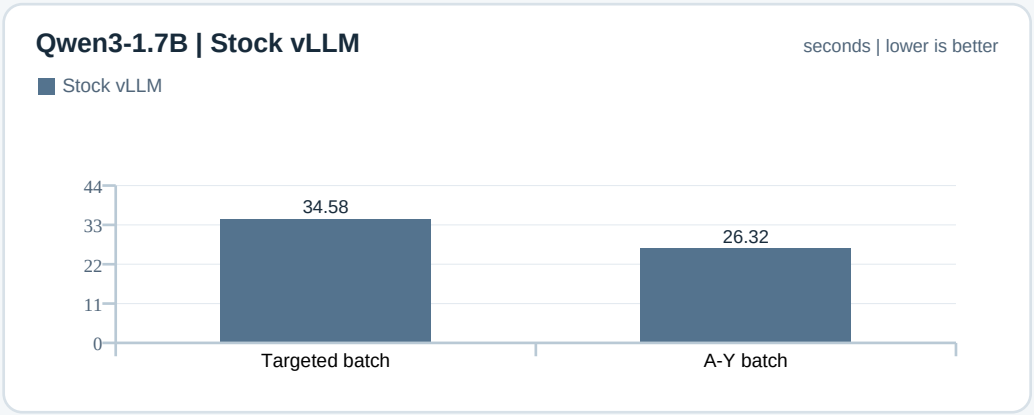
Retain cache that waiting requests need

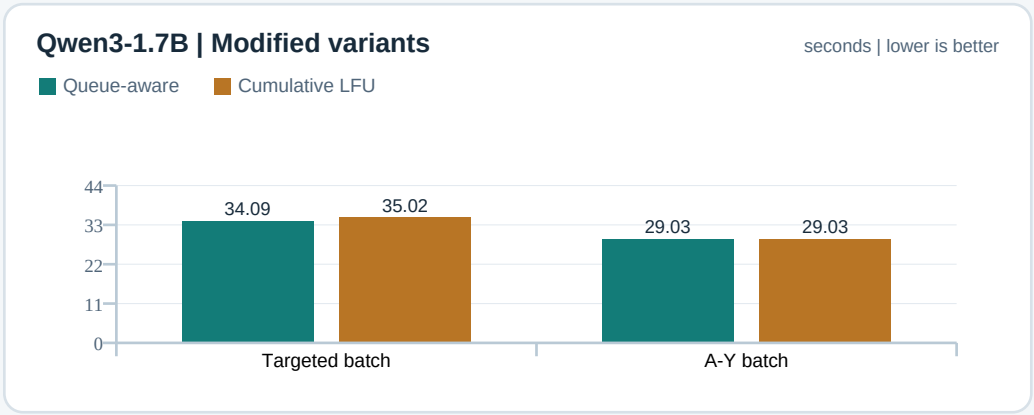
What changed. Use prefixes of already waiting requests to prefer retaining relevant zero-reference cached blocks. Keep FCFS request order and allow normal reclamation rather than permanently reserving pages.

Targeted return-prefix and original A-Y profiles | 3 policies | 24 runs/model | 3 rounds/policy/profile









Observed result. First-return reuse increased from 80 stock tokens (16 under LFU) to 176 queue-aware tokens on both models. Targeted batch median fell 9.82% on 0.6B and 1.42% on 1.7B.

Interpretation. On A-Y, every policy reused the same 2,432 tokens. Queue-aware 1.7B A-Y was 10.30% slower and lost all three stock pairs. Retention generalizes; a consistent speed advantage does not.

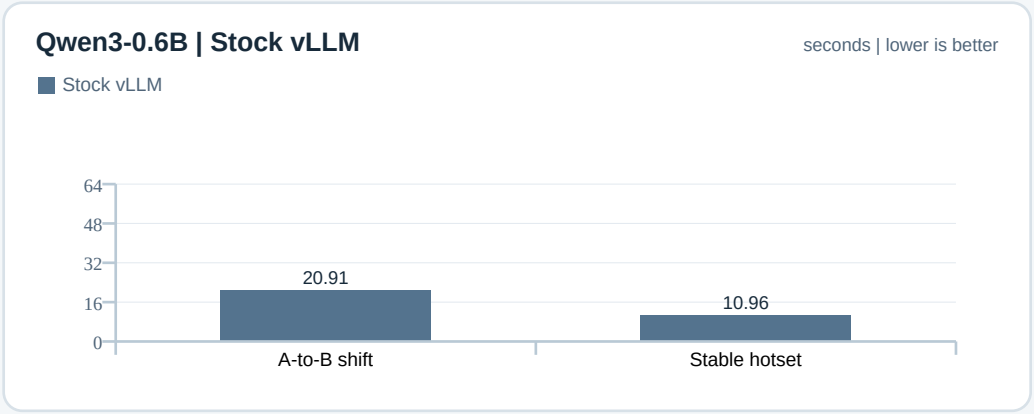
Sources: experiments/08-queue-retention/RESULTS.txt | experiments/08-queue-retention/qwen3-1.7B/RESULTS.txt

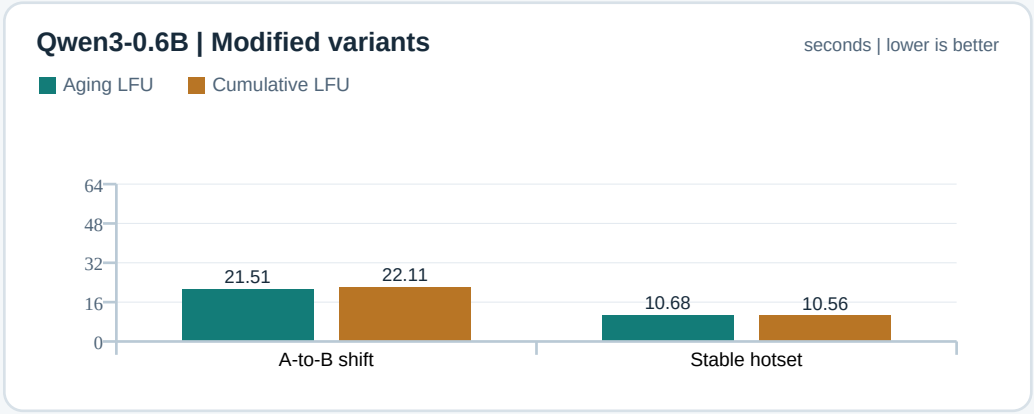
MacBook Air M5 / Metal | 3 timing rounds/configuration | separate model sessions | same axis across four panels; workloads differ by page

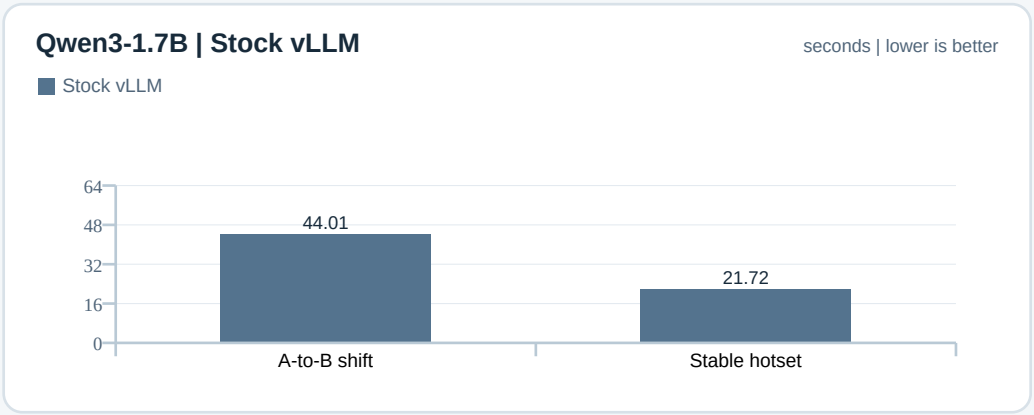
Let old cache popularity fade

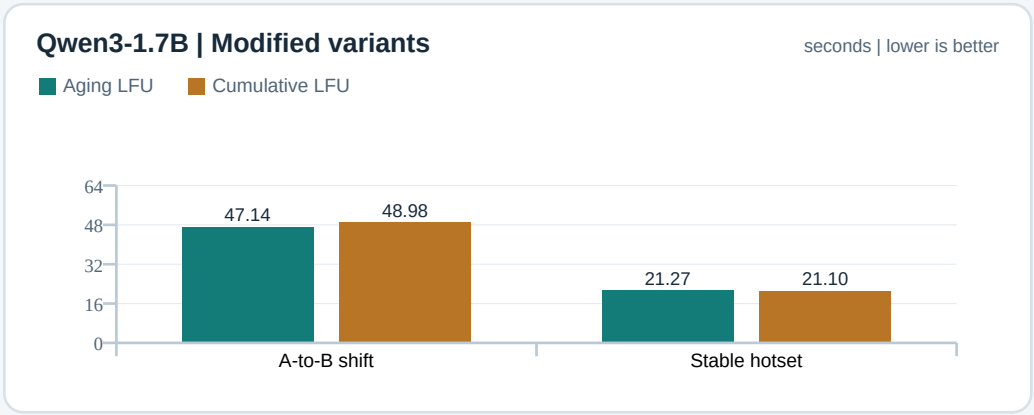
What changed. Halve all reuse scores every 64 physical-block allocations, including scores of active blocks. Prefer low-score zero-reference victims and preserve upstream free-queue tie order. Compare against stock and cumulative LFU.

84-request A-to-B shift and 42-request stable control | 32 pages | 1 slot | 24 runs/model | 3 rounds









Observed result. After cold scans, aging retained all 240 B-prefix tokens from probe cycle 2 on both models; stock retained 16 and cumulative LFU 32. Shift-probe first text improved 38.7% on 0.6B and 61.6% on 1.7B.

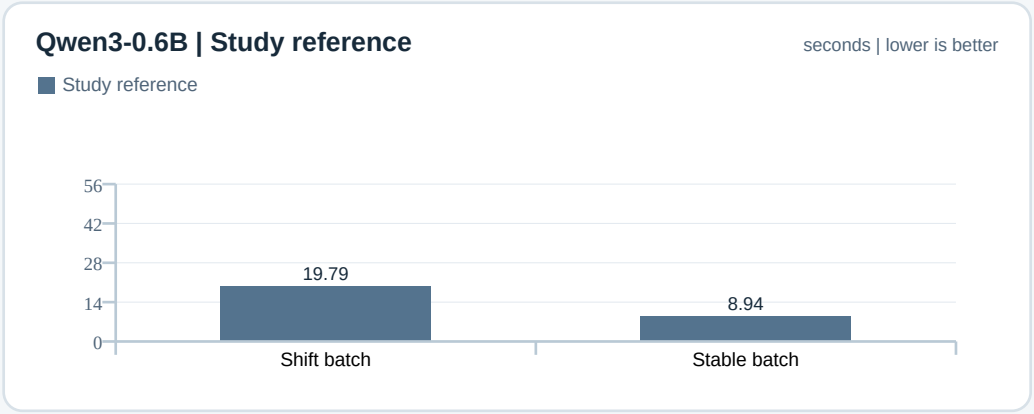
Interpretation. Shift batch group medians were slower than stock by 2.86%/7.10%, despite faster return probes. Stable retention stayed at 272 tokens for both LFU policies. All timed texts matched stock.

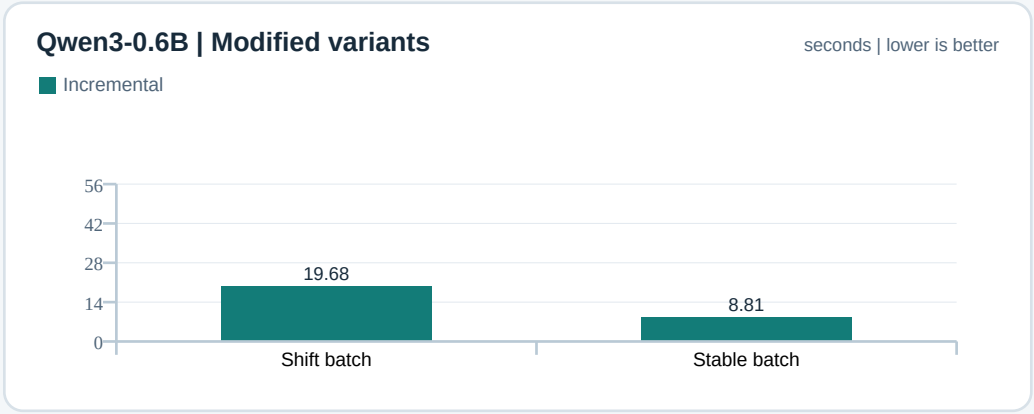
Sources: experiments/09-aging-lfu/RESULTS.txt | experiments/09-aging-lfu/qwen3-1.7B/RESULTS.txt

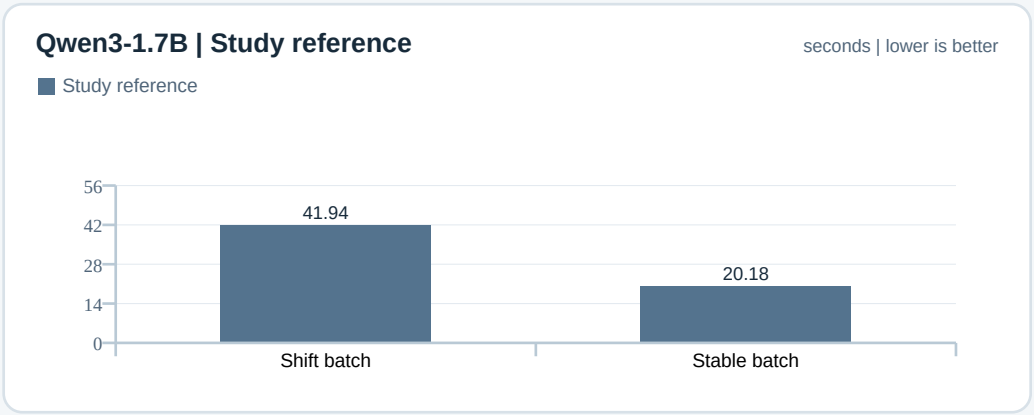
Reduce cache implementation overhead

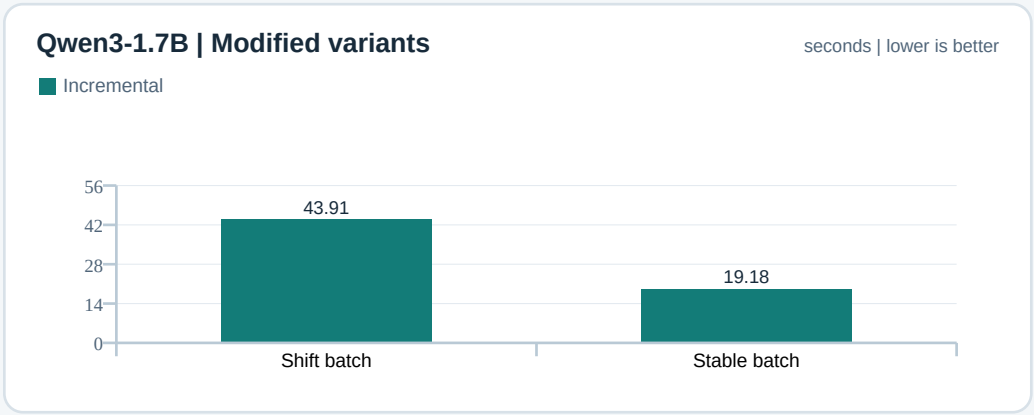
What changed. Publish only newly computed or invalidated prompt ranges instead of rescanning prompt blocks after every step. Replace repeated free-list sorting with a lazy-invalidated LFU heap, preserving selection ties, copies and lifecycle updates.

Matched reference vs incremental LFU + partial reuse | both models | shift/stable | 16 GPU runs/model | 3 pairs/profile









Observed result. Batch-median changes (shift / stable): 0.6B -0.56% / -1.46%; 1.7B +4.68% / -4.97%. On 1.7B, selector host time changed 7.53 to 5.65ms (shift) and 2.67 to 1.55ms (stable).

Interpretation. The control is the prior study LFU + partial-prefix implementation, not stock vLLM. Decisions, per-request cached counts and answers matched in both models. Less bookkeeping need not produce a large wall-time gain. Host timers include warmup/drains; the 4,096-page CPU replay is separate from GPU throughput.

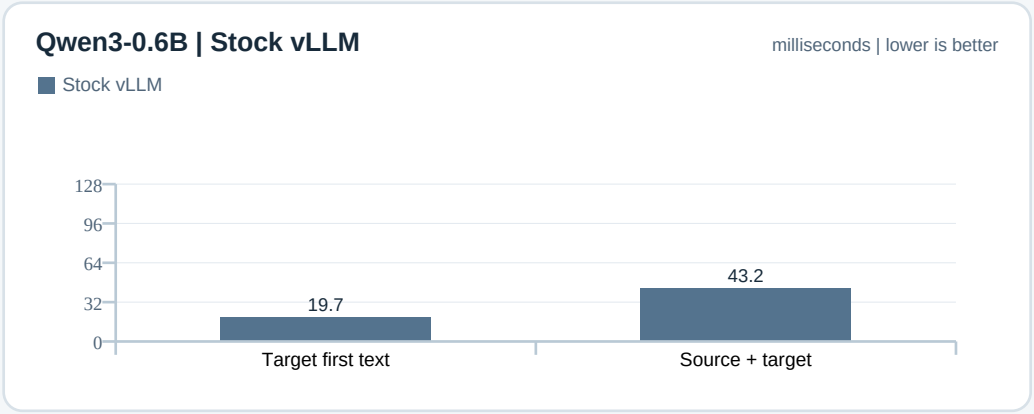
Sources: experiments/10-cache-overhead/RESULTS.txt | experiments/10-cache-overhead/qwen3-1.7B/RESULTS.txt

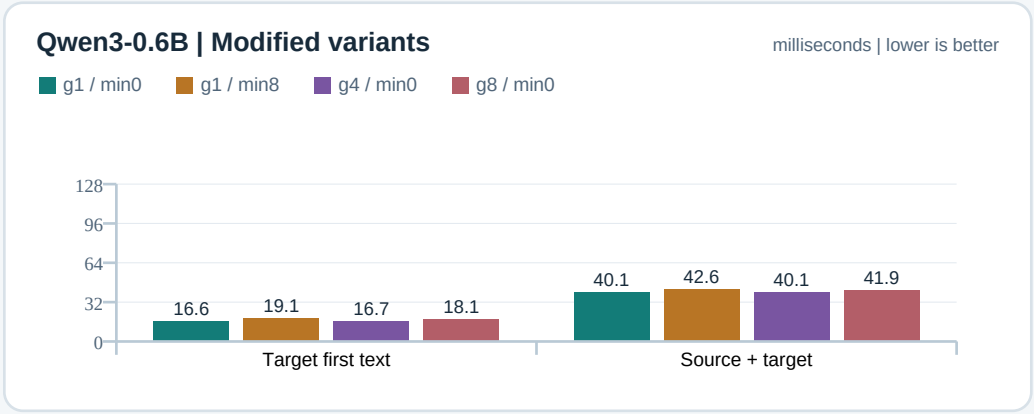
MacBook Air M5 / Metal | 3 timing rounds/configuration | separate model sessions | same axis across four panels; workloads differ by page

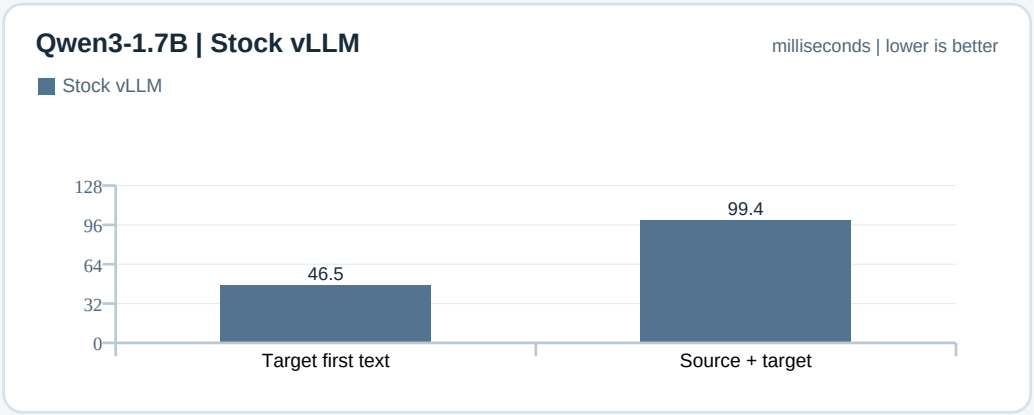
Partial-reuse thresholds and granularity

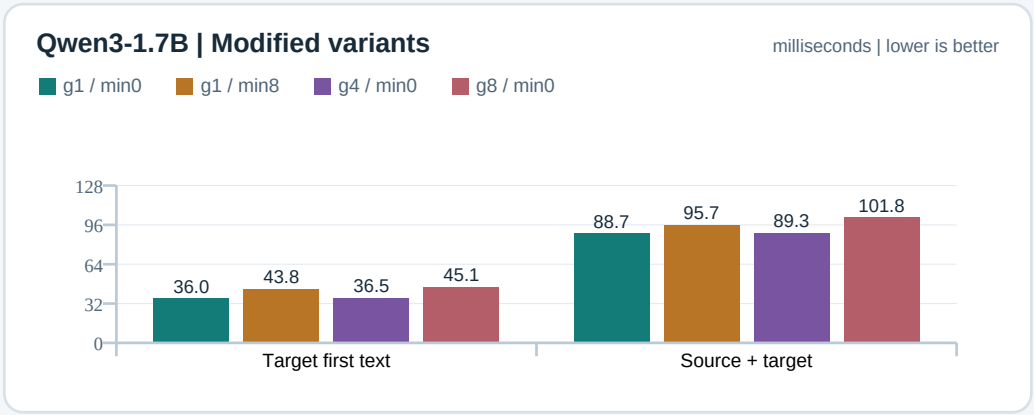
What changed. Accept partial hits only when they save a configured minimum number of extra token computations. Sweep 1/4/8-token matching and minimum 0/8/16; physical pages remain 16 tokens. min16 is a same-granularity recomputation control.

15 divergence offsets | 128-token prompts | 1 output/source and target | 135 timed targets/config | 40 runs/model









Observed result. Unrestricted g1 matching reduced target first text by 15.5% on 0.6B and 22.7% on 1.7B versus stock. Source+target time fell 7.3%/10.8%. The eight-token minimum discarded useful small hits.

Interpretation. Charts show stock and four of ten measured configurations; same-granularity min16 and other threshold controls remain in the source data. These are one-output-token probes, not concurrent throughput results. All 135 answers/config matched the recomputation control.

Sources: experiments/11-partial-threshold/RESULTS.txt | experiments/11-partial-threshold/qwen3-1.7B/RESULTS.txt

Partial reuse under concurrent mixed traffic

What changed. Carry exact one-token partial reuse into a mixed workload: unrestricted hits versus minimum eight saved tokens. Vary client concurrency while keeping four server slots and the same finite KV capacity. No new source policy is added.

1.7B only | 48 requests | 64-512 prompt / 16-64 output tokens | 16 seeded sources | 4/12 clients | 24 runs

Qwen3-0.6B | Original / control

NOT MEASURED

No saved matched measurements for this model. Values are not estimated or borrowed from another experiment.

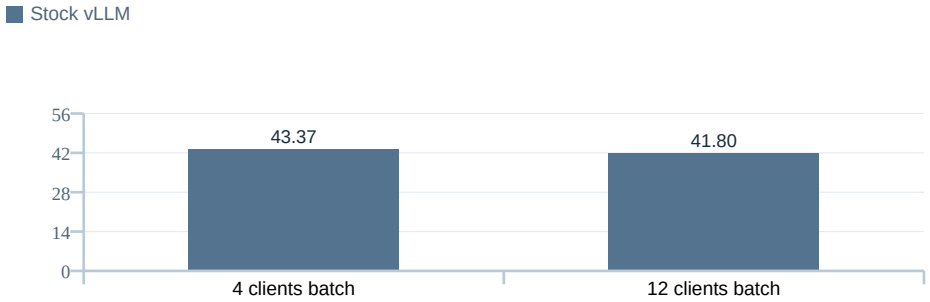
Qwen3-0.6B | Modified variants

NOT MEASURED

No saved matched measurements for this model. Values are not estimated or borrowed from another experiment.

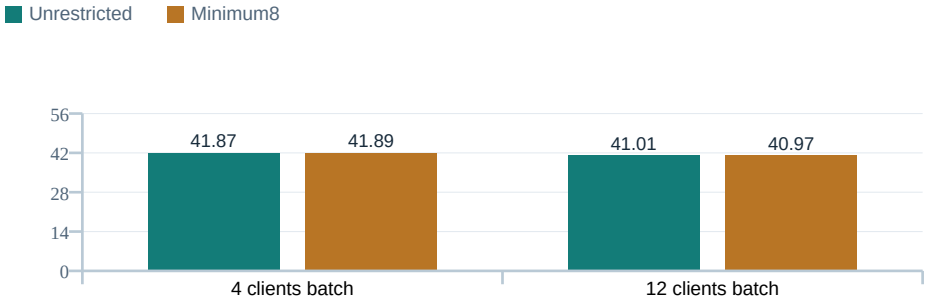
Qwen3-1.7B | Stock vLLM

seconds | lower is better



Qwen3-1.7B | Modified variants

seconds | lower is better



Observed result. Unrestricted batch medians improved 3.46% at four clients and 1.90% at twelve; minimum8 was nearly tied. At twelve clients, unrestricted p95 request latency worsened 4.81%. All paired texts matched stock.

Interpretation. Closed-loop arrivals depend on completions; the cache is seeded but churns. Seed time is excluded. Gains were not consistent in every round, and concurrency alone is not isolated from mixed lengths, longer decoding and finite-cache effects. No 0.6B replication was run.

Sources: experiments/12-concurrent-partial/RESULTS.txt

Adapt prefill chunks to current traffic

What changed. Compare stock chunked prefill with built-in fair-share adaptation and a new decoder-load cap: max(16,128/(1+2D)), clamped to the budget. An unfinished prompt older than 750ms raises the global per-request cap to at least 64.

1.7B only | 4 ongoing generators + 2 bursts of 10 long prompts | 8 slots | 128-token budget | caching/skip off | 12 runs

Qwen3-0.6B | Original / control

NOT MEASURED

No saved matched measurements for this model. Values are not estimated or borrowed from another experiment.

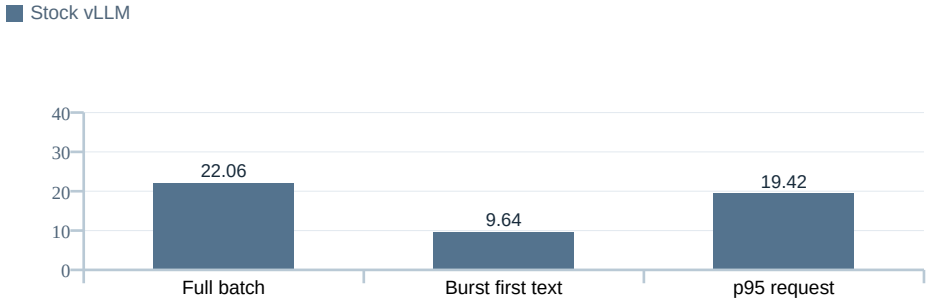
Qwen3-0.6B | Modified variants

NOT MEASURED

No saved matched measurements for this model. Values are not estimated or borrowed from another experiment.

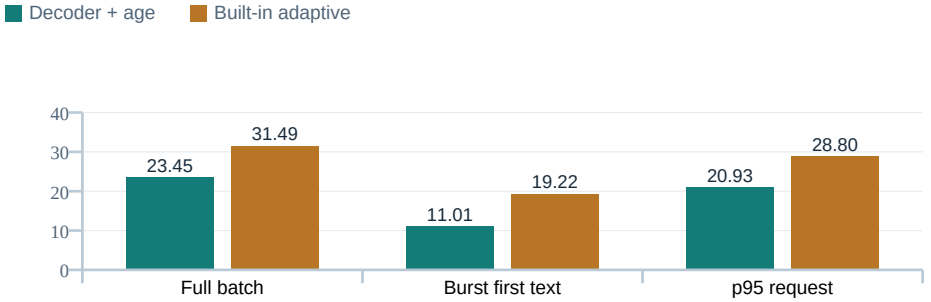
Qwen3-1.7B | Stock vLLM

seconds | lower is better



Qwen3-1.7B | Modified variants

seconds | lower is better



Observed result. The new cap reduced ongoing p95 text-chunk gaps from 201ms to 169ms (16%), but batch time rose 6.3% and burst first text 14.2%. Built-in adaptation reduced gaps to 145ms while batch time rose 42.8%.

Interpretation. This is a streaming smoothness tradeoff, not a speed win. All timed texts matched stock. Chunk gaps are not exact token ITL; the age boost is not a strict waiting bound. Three fresh-server rounds with one short warmup are exploratory.

Sources: experiments/13-adaptive-prefill/RESULTS.txt